Mi etapa del flujo de FinNorte

Kevin Morales · KevinAdmp · Equipo 2 · v2-revisor

1. Mi etapa

Mi etapa es **dictamen**, con id `rev_dictamen`, la cuarta del flujo detectar, explicar, parche, dictamen. Recibe con `usa=["explicar", "parche"]` los hallazgos explicados y los parches propuestos, y entrega estado, total, hallazgos con su cambio y resumen. Ayuda porque quien revisa lee un solo reporte: los hallazgos, «sin hallazgos» o un aviso de revisión fallida. Es la última etapa, así que su salida alimenta ese reporte final, que arma el programa.

2. Mi prompt v1

`prompts/rev_dictamen.v1.md` usa rol, descomposición en pasos numerados, restricciones, criterios de validación, formato JSON con ejemplo y manejo de faltantes. Su esquema `rev_dictamen.schema.json` es estricto (`additionalProperties: false`). Ejemplo: «Copia sin cambios id, regla, linea, descripcion y explicacion del hallazgo, y cambio del parche».

3. Mi validación en código

`validar_dictamen` en `src/finnorte/dictamen.py` compara el reporte con las salidas reales de explicar y parche: rechaza hallazgos omitidos, inventados o repetidos, campos cambiados, estado o total incoherentes, datos ajenos en el resumen, credenciales y propuestas presentadas como aplicadas. Es código porque la fidelidad se comprueba comparando datos, y el modelo no puede evaluarse a sí mismo.

Caso real: en `credencial.py` (`comparar_v1_regla_resumen`, caso d3), v1 escribió «como medida de mitigación». El validador respondió: «El resumen agrega texto que no está en los datos recibidos: detalla, medida, mitigacion, propone, reporte, variables». El reintento pasó. Las reglas de copia exacta no se dispararon en ejecuciones reales; solo se comprobaron en pruebas sin red, igual que el aviso de revisión fallida.

4. Evidencia

Extracto de `bitacora/dictamen/comparar_v2/trazas.jsonl`:

```
prompt: rev_dictamen.v2 · modelo: gemini-3.1-flash-lite (gemini)
fecha UTC: 2026-11-06T02:27:27 · caso d6 · intento 1 · ok: True
```

Fila de `bitacora/dictamen/comparar_v2/matriz.md`:

```
d6 | inicial | dos_problemas.py | esperado SEC-01, SEC-02 | detectado SEC-01, SEC-02
| hallazgos | coincide True | 1 intento | primer intento válido True
```

5. Consistencia

`dos_problemas.py` se repitió 3 veces. Con v1 y v2 las tres dieron SEC-01 en línea 1 y SEC-02 en línea 5, y reporte con hallazgos. La diferencia estuvo en el resumen: 3 redacciones distintas con v1 y 1 con v2. Coincidir no demuestra exactitud; por eso cada caso también se compara con lo esperado.

6. Mi mejora v2

Fallo real: la v1 pasó 12/12 con mi primer validador, pero al leer sus resúmenes vi afirmaciones que nadie le entregó («seguridad de la información», «mitigar la vulnerabilidad») y una redacción distinta en cada corrida. Agregué esa regla y repetí la v1 sin editarla. La causa era que «dos frases» dejaba libre el texto, así que la v2 fija una plantilla y prohíbe describir las reglas. El motivo está en su campo `cambios:`.

Medida, 6 flujos, mismas entradas y validador	v1	v2
Válido al primer intento	1/6	6/6
Redacciones en 3 repeticiones	3	1
Reporte correcto	6/6	6/6

En los seis casos pedidos (dos limpios, tres con un problema y uno con dos), v2 dio 6/6. Límite: son archivos pequeños y pocas corridas, y con plantilla el resumen podría generarlo el código; lo que aporta el modelo es unir hallazgos y parches.

7. Mis commits y uso de IA

Commits: https://github.com/alanrivera066/Project_v2_TSTI/commits/etapa4-dictamen

Usé Claude Code con Claude Opus 5.5 para redactar prompts, código, pruebas y este borrador. El registro está en `ai_log.md`.